

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems. (L4, L5)

Assessment Tool : Industry Problem Task

Weightage: 40%

QUESTIONS:

Total Marks: 30

1. A hospital management system requires a **SOAP-based web service** to manage patient information such as registration, appointment scheduling, and medical records. The system must ensure secure, structured, and interoperable communication between different hospital departments.

Design a **SOAP-based web service** for the hospital system. Explain the structure of SOAP messages (Envelope, Header, Body), define the operations involved (e.g., add patient, get records), and illustrate how data is exchanged between client and server. Justify the use of SOAP in this scenario considering reliability and security.

2. An enterprise application needs to expose its web services so that external systems can understand how to interact with them. The organization plans to use **WSDL (Web Services Description Language)** to describe the service.

Design the **WSDL structure** for a web service that provides operations such as retrieving and updating data. Explain the key components of WSDL (types, message, portType, binding, service), and illustrate how they define the service interface and communication details. Provide a clear structure and justify its importance in service integration.

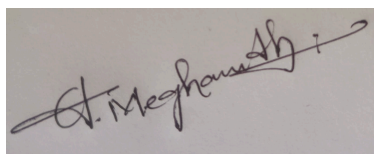
3. A company is experiencing issues in its SOAP-based communication system where requests are not processed correctly, and responses are either delayed or incorrect.

Analyze the SOAP communication process and identify possible issues such as message formatting errors, incorrect namespaces, or endpoint problems. Propose a debugging approach to resolve these issues, including validation of SOAP messages and error handling techniques. Explain how the corrected system ensures reliable communication.

Code of Conduct:

I _____A.Meghanadh(192372281)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:**RUBRICS FOR CALCULATION:**

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

SOAP-Based Hospital Management Web Service

A **SOAP (Simple Object Access Protocol) web service** is suitable for a hospital management system because it provides structured, standardized, and secure communication between different hospital departments such as reception, outpatient services, laboratory, pharmacy, and medical records.

1. Proposed Architecture

The system can use the following architecture:

Hospital Client → SOAP Request → Web Service → Hospital Database

Hospital Client ← SOAP Response ← Web Service

The SOAP service can expose a WSDL (Web Services Description Language) document that describes its operations, input/output messages, and service endpoint.

2. SOAP Message Structure

Every SOAP message consists mainly of an **Envelope**, with optional **Header** information and a **Body**.

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
```

```
  <soap:Header>
```

```
    <!-- Security, authentication, transaction information -->
```

```
  </soap:Header>
```

```
  <soap:Body>
```

```
    <!-- Request or response data -->
```

```
  </soap:Body>
```

```
</soap:Envelope>
```

a) Envelope

The **Envelope** is the outermost element. It identifies the message as a SOAP message and contains the Header and Body.

b) Header

The **Header** is optional and contains information that applies to the message as a whole. In a hospital system it can contain:

- Authentication credentials or security tokens
- Digital signatures
- Encryption information
- Transaction identifiers
- Message-routing information

For example:

```
<soap:Header>
  <SecurityToken>ABC123XYZ</SecurityToken>
  <TransactionID>TXN10025</TransactionID>
</soap:Header>
```

c) Body

The **Body** contains the actual request or response. For example, an **AddPatient** request could contain:

```
<soap:Body>
  <AddPatient>
    <Patient>
      <PatientID>P1001</PatientID>
      <Name>John Smith</Name>
      <DateOfBirth>1990-05-15</DateOfBirth>
      <Gender>Male</Gender>
      <Contact>9876543210</Contact>
    </Patient>
  </AddPatient>
</soap:Body>
```

3. Main Web-Service Operations

The hospital SOAP service could provide operations such as:

Operation	Purpose
AddPatient	Register a new patient

GetPatient Retrieve patient demographic information

UpdatePatient Modify patient information

ScheduleAppointment Create an appointment

CancelAppointment Cancel an appointment

GetAppointments Retrieve scheduled appointments

AddMedicalRecord Add diagnosis, treatment, or test information

GetMedicalRecords Retrieve a patient's medical records

UpdateMedicalRecord Update an existing medical record

4. Example: Adding a Patient

The registration department sends a SOAP request to the hospital service:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header>
    <SecurityToken>ABC123XYZ</SecurityToken>
```

```
</soap:Header>

<soap:Body>
  <AddPatient>
    <Patient>
      <PatientID>P1001</PatientID>
      <Name>John Smith</Name>
      <DateOfBirth>1990-05-15</DateOfBirth>
      <Gender>Male</Gender>
    </Patient>
  </AddPatient>
</soap:Body>
</soap:Envelope>
```

The web service validates the request, stores the patient information in the hospital database, and returns a SOAP response:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <AddPatientResponse>
      <Status>Success</Status>
      <PatientID>P1001</PatientID>
      <Message>Patient registered successfully</Message>
    </AddPatientResponse>
  </soap:Body>
</soap:Envelope>
```

5. Example: Getting Medical Records

A doctor can request the medical records of patient P1001:

```
<soap:Body>  
  <GetMedicalRecords>  
    <PatientID>P1001</PatientID>  
  </GetMedicalRecords>  
</soap:Body>
```

The server processes the request, retrieves authorized records from the database, and sends them back:

```
<soap:Body>  
  <GetMedicalRecordsResponse>  
    <PatientID>P1001</PatientID>  
    <Record>  
      <Date>2026-08-20</Date>  
      <Diagnosis>Example diagnosis</Diagnosis>  
      <Treatment>Example treatment</Treatment>  
    </Record>  
  </GetMedicalRecordsResponse>  
</soap:Body>
```

Thus, different departments can communicate with the same service without needing to know how the underlying database is implemented.

6. Error Handling

SOAP provides a standard **Fault** mechanism for reporting errors.

For example:

```
<soap:Body>  
  <soap:Fault>  
    <faultcode>Client</faultcode>
```

```
<faultstring>Patient ID is required</faultstring>

</soap:Fault>

</soap:Body>
```

This allows clients to handle invalid requests, authentication failures, unavailable services, and other errors consistently.

7. Why SOAP is Appropriate

Security: Hospital systems handle sensitive patient information. SOAP supports standards such as **WS-Security**, allowing message-level authentication, digital signatures, and encryption. HTTPS can additionally protect communication while it is in transit.

Reliability: SOAP-based web services can use WS-* standards for features such as reliable messaging, transactions, and standardized fault handling. This is useful when important hospital operations must be delivered consistently.

Interoperability: SOAP uses XML and standardized contracts such as WSDL, allowing applications written in different programming languages and running on different platforms to communicate.

Structured communication: The Envelope/Header/Body structure provides a clearly defined format. Security and transaction information can be placed in the Header while business data remains in the Body.

Enterprise integration: Hospitals often have multiple legacy and modern systems. SOAP is well suited to integrating heterogeneous enterprise applications where formal contracts, security, and reliable messaging are important.

Conclusion

A SOAP-based hospital management service can expose operations such as **AddPatient**, **ScheduleAppointment**, and **GetMedicalRecords** through a formally defined WSDL interface. Clients send structured XML SOAP requests, the service validates and processes them against the hospital database, and SOAP responses return the results. **SOAP is particularly appropriate where interoperability, strong security, standardized error handling, and reliable enterprise communication are more important than the simplicity and lightweight nature of REST.**

2. WSDL Structure for an Enterprise Web Service

WSDL (Web Services Description Language) is an XML-based language used to formally describe a web service. It tells external applications **what operations are available, what data must be exchanged, how communication takes place, and where the service is located.**

A typical WSDL document contains five important components:

1. **Types**
2. **Message**
3. **PortType**
4. **Binding**
5. **Service**

1. Types

The **types** section defines the data structures used by the service. XML Schema (XSD) is commonly used.

For example, a service that retrieves and updates customer data could define:

<types>

<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">

<xsd:element name="GetCustomerRequest">

<xsd:complexType>

<xsd:sequence>

<xsd:element name="CustomerID" type="xsd:int"/>

</xsd:sequence>

</xsd:complexType>

</xsd:element>

<xsd:element name="UpdateCustomerRequest">

<xsd:complexType>

```
<xsd:sequence>
  <xsd:element name="CustomerID" type="xsd:int"/>
  <xsd:element name="Name" type="xsd:string"/>
  <xsd:element name="Email" type="xsd:string"/>
</xsd:sequence>
</xsd:complexType>
</xsd:element>

</xsd:schema>
</types>
```

The **types** section therefore specifies **the structure and data types of the information being exchanged**.

2. Message

The **message** element defines the logical messages exchanged between the client and service.

```
<message name="GetCustomerInput">
  <part name="parameters" element="tns:GetCustomerRequest"/>
</message>

<message name="GetCustomerOutput">
  <part name="parameters" element="tns:GetCustomerResponse"/>
</message>

<message name="UpdateCustomerInput">
  <part name="parameters" element="tns:UpdateCustomerRequest"/>
</message>
```

A message can contain one or more **part** elements representing the data being transmitted.

3. PortType

portType defines the **operations offered by the web service**. It is similar to an interface in programming.

```
<portType name="CustomerPortType">

    <operation name="GetCustomer">

        <input message="tns:GetCustomerInput"/>

        <output message="tns:GetCustomerOutput"/>

    </operation>

    <operation name="UpdateCustomer">

        <input message="tns:UpdateCustomerInput"/>

        <output message="tns:UpdateCustomerOutput"/>

    </operation>

</portType>
```

Here, the service provides two operations:

- **GetCustomer** — retrieves customer information.
- **UpdateCustomer** — modifies customer information.

4. Binding

The **binding** section specifies **how the operations are actually transmitted**. For a SOAP service, it can specify SOAP over HTTP.

```
<binding name="CustomerSoapBinding"

    type="tns:CustomerPortType">
```

```
<soap:binding
  style="document"
  transport="http://schemas.xmlsoap.org/soap/http"/>

<operation name="GetCustomer">
  <soap:operation soapAction="GetCustomer"/>
  <input>
    <soap:body use="literal"/>
  </input>
  <output>
    <soap:body use="literal"/>
  </output>
</operation>

<operation name="UpdateCustomer">
  <soap:operation soapAction="UpdateCustomer"/>
  <input>
    <soap:body use="literal"/>
  </input>
  <output>
    <soap:body use="literal"/>
  </output>
</operation>
</binding>
```

Thus, `portType` says **what the service does**, while `binding` says **how those operations are communicated**.

5. Service

The `service` element specifies the actual endpoint where the service can be accessed.

```
<service name="CustomerService">

    <port name="CustomerSoapPort"
          binding="tns:CustomerSoapBinding">

        <soap:address
              location="https://example.com/customerService"/>
    </port>

</service>
```

The client can use this endpoint to send SOAP requests.

Complete WSDL Structure

A simplified WSDL document can therefore be represented as:

```
<definitions

    xmlns="http://schemas.xmlsoap.org/wsdl/"
    xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
    xmlns:xsd="http://www.w3.org/2001/XMLSchema"
    xmlns:tns="http://example.com/customer"
    targetNamespace="http://example.com/customer">

    <types>
```

```
<!-- XML Schema definitions -->

</types>

<message name="GetCustomerInput">
    <!-- Request data -->
</message>

<message name="GetCustomerOutput">
    <!-- Response data -->
</message>

<message name="UpdateCustomerInput">
    <!-- Update request data -->
</message>

<message name="UpdateCustomerOutput">
    <!-- Update response data -->
</message>

<portType name="CustomerPortType">
    <!-- GetCustomer and UpdateCustomer operations -->
</portType>

<binding name="CustomerSoapBinding"
    type="tns:CustomerPortType">
```

```
<!-- SOAP communication details -->

</binding>

<service name="CustomerService">

  <port name="CustomerSoapPort"

    binding="tns:CustomerSoapBinding">

      <soap:address

        location="https://example.com/customerService"/>

      </port>

    </service>

  </definitions>
```

Importance of WSDL in Service Integration

WSDL is important because it provides a **machine-readable contract** between service providers and consumers.

- **Standardization:** Clearly defines the service interface.
- **Interoperability:** Different programming languages and platforms can communicate.
- **Reduced development effort:** Tools can generate client/server code from a WSDL document.
- **Clear contract:** Clients know the available operations, input/output formats, and endpoint.
- **Loose coupling:** Clients depend on the service contract rather than its internal implementation.
- **Easy integration:** External systems can discover and consume the service using the WSDL definition.

Therefore, WSDL acts as a **formal agreement between a service provider and service consumer**, making enterprise service integration predictable and reliable.

3. Debugging Problems in SOAP-Based Communication

When SOAP requests are not processed correctly or responses are delayed or incorrect, the problem can occur at several levels: **SOAP message construction, namespaces, transport, endpoint configuration, server processing, or error handling.**

1. Analyze the SOAP Communication Process

A normal SOAP communication process is:

Client → Create SOAP Request → HTTP/HTTPS → SOAP Endpoint → Validate Request → Process Operation → Generate SOAP Response → Client

For example:

Client

|

| SOAP Request

v

HTTP/HTTPS Endpoint

|

v

SOAP Web Service

|

| Validate + Process

v

Database/Application

|

| Result

v

SOAP Response

|

v

Client

The first debugging step is to identify **where the communication fails**.

2. Check SOAP Message Formatting

SOAP messages must follow the expected XML structure.

A valid request should contain an Envelope and Body:

```
<soap:Envelope
```

```
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
```

```
  <soap:Header>
```

```
    <!-- Optional header information -->
```

```
  </soap:Header>
```

```
  <soap:Body>
```

```
    <GetCustomer xmlns="http://example.com/customer">
```

```
      <CustomerID>1001</CustomerID>
```

```
    </GetCustomer>
```

```
  </soap:Body>
```

```
</soap:Envelope>
```

Possible formatting problems include:

- Missing SOAP Envelope.
- Incorrect XML syntax.
- Incorrect element names.
- Missing required parameters.
- Incorrect data types.
- Malformed XML.
- Unexpected elements.
- Incorrect SOAP version.

The message should first be validated as **well-formed XML**, followed by validation against the service's XML Schema/WSDL.

3. Check Namespaces

Namespaces are particularly important in SOAP because the server uses them to identify XML elements.

For example:

```
<GetCustomer xmlns="http://example.com/customer">
```

If the service expects:

<http://example.com/customer>

but the client sends:

<http://example.org/customer>

the server may not recognize the operation.

Common namespace errors include:

- Wrong namespace URI.
- Missing namespace declaration.
- Incorrect namespace prefix.
- Using the namespace of another service.
- Incorrect SOAP Envelope namespace.

The namespace URI, not merely the prefix name, must match the service definition

4. Verify the Endpoint

The client must send the request to the correct SOAP endpoint.

For example:

<https://example.com/customerService>

https://example.com/customerService

Possible endpoint problems include:

- Incorrect URL.
- Server unavailable.
- Incorrect port.
- DNS problems.
- Firewall blocking the connection.
- Incorrect HTTP/HTTPS configuration.
- Service deployed at a different path.
- Network timeout.

The endpoint should be tested independently to determine whether the service is reachable.

5. Check SOAPAction and HTTP Headers

Some SOAP services require an appropriate **SOAPAction** HTTP header.

For example:

POST /customerService HTTP/1.1

Host: example.com

Content-Type: text/xml; charset=utf-8

SOAPAction: "GetCustomer"

An incorrect or missing SOAPAction can cause the server to invoke the wrong operation or reject the request.

The **Content-Type** should also correspond to the SOAP version being used.

6. Validate the WSDL and XML Schema

The WSDL should be checked to confirm:

- Correct operation name.

- Correct input message.
- Correct output message.
- Correct namespace.
- Correct binding.
- Correct endpoint.
- Correct data types.

The request should then be compared against the WSDL/XSD definitions.

For example, if the schema expects:

```
<CustomerID>1001</CustomerID>
```

but the client sends:

```
<CustomerID>ABC</CustomerID>
```

the request may fail because the expected type is numeric.

7. Examine SOAP Faults

SOAP provides a standard mechanism called **SOAP Fault** for communicating errors.

Example:

```
<soap:Fault>
  <faultcode>soap:Client</faultcode>
  <faultstring>Invalid CustomerID</faultstring>
  <detail>
    <ErrorCode>4001</ErrorCode>
  </detail>
</soap:Fault>
```

The client should not simply treat every response as successful. It should check for a SOAP Fault and provide appropriate error handling.

Typical fault categories include:

- **Client errors:** Invalid request or parameters.
- **Server errors:** Internal application failure.
- **Security errors:** Authentication or authorization failure.

- **Processing errors:** Database or business-logic problems.
-

8. Investigate Delayed Responses

If responses are delayed, investigate both the network and server.

Possible causes include:

1. Network latency.
2. Server overload.
3. Slow database queries.
4. Connection timeouts.
5. Incorrect retry configuration.
6. Deadlocks or resource contention.
7. External service dependencies.

Server logs should be examined to determine how long each stage takes.

A **correlation/transaction ID** can also be included in SOAP headers so that a particular request can be traced across multiple systems.

9. Recommended Debugging Procedure

A systematic approach can be followed:

1. Reproduce the problem

↓

2. Capture the SOAP request and response

↓

3. Validate XML syntax

↓

4. Validate against WSDL/XSD

↓

5. Check SOAP Envelope and SOAP version



6. Check namespaces



7. Check operation name and SOAPAction



8. Verify endpoint and HTTP/HTTPS connection



9. Examine SOAP Faults and server logs



10. Check database/business logic



11. Test again with a known-good request

Tools such as SOAP testing clients, HTTP tracing tools, server logs, and XML validators can be used to inspect the request/response exchange.

10. Ensuring Reliable Communication

After identifying the problem, the corrected system should implement:

- **Strict XML and schema validation** before processing requests.
- **Correct namespaces** based on the WSDL.
- **Correct endpoint and SOAPAction configuration.**
- **Standard SOAP Fault responses** for errors.
- **Meaningful server-side logging.**
- **Correlation IDs** for request tracing.
- **Timeout and controlled retry mechanisms** for transient failures.
- **HTTPS and appropriate authentication** to protect communication.
- **Input validation** to prevent invalid data from reaching business logic.
- **Automated tests** for each SOAP operation.

Conclusion

SOAP communication depends on several components working together correctly: XML formatting, namespaces, WSDL definitions, HTTP headers, endpoint configuration, server processing, and error handling. A structured debugging process

should begin by capturing and validating the SOAP message, then checking namespaces and WSDL compliance, followed by endpoint, network, server, and database investigation. Proper **SOAP Fault handling, validation, logging, timeout/retry policies, and security mechanisms** ensure that the final system provides reliable and predictable communication between enterprise applications.